Day 3: Test Execution and Defect Management

Execution statuses, defect life cycle, Jira bug standards, severity vs priority, and regression testing.

1. Day Learning Objectives

[ ] Execute manual test cases and record accurate statuses (Pass, Fail, Blocked, Not-Run)
[ ] Trace the complete Defect Life Cycle from creation to closure
[ ] Author world-class, reproducible Jira bug reports developers can fix immediately
[ ] Accurately classify Defect Severity (technical impact) vs Priority (business urgency)
[ ] Differentiate Retesting (verifying bug fix) vs Regression Testing (preventing side-effects)
[ ] Distinguish Smoke Testing (build stability) vs Sanity Testing (focused functional check)
[ ] Gather and communicate defect evidence (logs, screenshots, HAR files) professionally

2. Core Concepts & Practical Fundamentals

1. Test Execution & Result Categorization

During test execution, each test case is executed against the application build and assigned one of four standard statuses:
• Pass: Actual outcome matches expected outcome 100%.
• Fail: Actual outcome differs from expected outcome; a defect ticket must be logged.
• Blocked: A prerequisite bug or environment issue prevents executing the test steps.
• Not-Run: Scheduled test case that has not yet been executed in the current test cycle.

Real-World Workplace Examples:
• **[Payment Gateway Integration]:** Test case "Verify receipt email after credit card payment": The credit card checkout button crashes with 500 error. Test status is FAIL for checkout, and receipt test is BLOCKED because checkout cannot complete.

Key Standard Terminology:
• **Actual Result:** The observed behavior produced by the software under test during execution.
• **Blocked Test:** A test case that cannot be executed due to an upstream blocking defect or missing environment dependency.

Common Beginner Mistakes & Interview Traps:
& Marking a test as "Pass" when a minor error was observed that the tester assumed was unimportant.
& Marking a test as "Fail" when the failure was caused by tester misconfiguration rather than software defect.

2. The Defect Life Cycle & Jira Standards

The Defect Life Cycle represents the states a bug passes through:
New -> Assigned -> Open -> Fixed -> Retest -> Closed.
Alternative paths: Rejected (not a bug), Duplicate (already logged), Deferred (postponed to future release), Reopened (bug reproduced upon retest).

A professional bug report contains:
1. Title: [Component/Module] Summary under condition
2. Environment: OS, Browser, App Build version
3. Preconditions: Required state prior to reproduction
4. Steps to Reproduce: Numbered, deterministic, minimal steps
5. Expected Result: What requirements state should happen
6. Actual Result: What system actually did (including error text)
7. Severity & Priority: Technical impact & business urgency
8. Evidence: Annotated screenshot, video recording, console logs

Real-World Workplace Examples:
• **[Shopping Cart Calculation Defect]:** Title: [Cart] Applying coupon "SAVE50" on subtotal under $50 causes negative order total balance.
Steps: 1. Add item ($35.00) to cart. 2. Enter promo "SAVE50". 3. Observe total shows -$15.00.
Expected: Displays error "Order must be at least $50.00". Actual: Total is -$15.00 and allows checkout.

Key Standard Terminology:
• **Defect Life Cycle:** The sequence of states that a defect goes through from discovery to resolution.
• **Reproducibility:** The ability to consistently recreate a defect following documented steps.

Common Beginner Mistakes & Interview Traps:
& Writing vague titles like "Checkout broken" without component or condition.
& Missing step numbers or combining multiple actions into a single confusing paragraph.

3. Severity vs Priority Matrix

• Severity (Technical Impact): The extent to which the defect impacts software operation, data integrity, or infrastructure (Blocker, Critical, Major, Minor).
• Priority (Business Urgency): How quickly the business needs the defect resolved based on release timeline and customer visibility (P0 - Immediate, P1 - High, P2 - Medium, P3 - Low).

Classic Matrices:
- High Severity, Low Priority: Rare crash in an annual tax report module executed only on December 31st.
- Low Severity, High Priority: Company CEO name or logo is misspelled on the public homepage hero banner on IPO launch day.

Real-World Workplace Examples:
• **[Crash on Logout]:** Application throws unhandled 500 error when clicking Logout, but user session is invalidated. High Severity (crash), Medium Priority.
• **[Typo in Legal Disclaimer]:** Spelling mistake in terms of service. Low Severity, High Priority (compliance deadline).

Key Standard Terminology:
• **Severity:** The degree of impact that a defect has on the development or operation of a system.
• **Priority:** The business importance and urgency assigned to fixing a defect.

Common Beginner Mistakes & Interview Traps:
& Confusing Severity and Priority in client meetings or job interviews.
& Assuming all crashes must automatically be assigned P0 immediate priority regardless of business context.

---

4. Retesting vs Regression Testing & Smoke vs Sanity

• Retesting: Testing specifically to verify that a reported bug has been fixed and no longer reproduces.
• Regression Testing: Testing unchanged parts of the application to ensure that bug fixes or code changes have not introduced unintended side effects.

• Smoke Testing (Build Verification): Broad, shallow test suite run on fresh builds to verify core stability before full QA execution starts.
• Sanity Testing: Narrow, deep test suite run after minor bug fixes to verify specific functionality works as expected.

Real-World Workplace Examples:
• **[Post-Bug-Fix Cycle]:** Developer fixes login lockout bug. QA performs Retesting on login lockout, then performs Regression Testing on password reset and registration.

Key Standard Terminology:
• **Retesting:** Executing test cases that previously failed to verify the defect was corrected.
• **Regression Testing:** Testing of a previously tested program to ensure that defects have not been introduced as a result of modifications.
• **Smoke Testing:** A subset of test cases covering primary functions to determine if a build is stable enough for deeper testing.

Common Beginner Mistakes & Interview Traps:
& Performing only retesting after a bug fix and skipping regression testing.
& Running a 6-hour regression suite when a 15-minute sanity test was appropriate for a minor CSS fix.

---

3. Guided Practice Exercise

Guided Exercise: Writing a Production-Grade Jira Bug Report

Observe a real application crash during payment submission and construct a Jira ticket following FAANG QA standards.

Feature Specification Under Test:

APPLICATION UNDER TEST: Shopping Cart Payment Flow
Scenario: User with valid cart items enters credit card details. When user clicks "Pay $99.00", the network request returns HTTP 500 Internal Server Error, button becomes disabled, and browser console displays: "Uncaught TypeError: Cannot read property 'currency' of undefined".

Recommended Execution Steps:
• Step 1: Write an unambiguous Title: [Checkout] Clicking Pay button throws 500 Internal Server Error on currency parameter.
• Step 2: Document Environment (Chrome 125, macOS Sonoma, Staging Build v2.4.1).
• Step 3: Number deterministic Steps to Reproduce.
• Step 4: Explicitly define Expected Result vs Actual Result.
• Step 5: Assign Severity (Critical) and Priority (High).

4. Practical Workplace Assignment & Evaluation Rubric

Day 3 Assignment: Execute Tests and Write 3 Professional Defect Reports

Review the provided test execution evidence and author at least 3 comprehensive, professional Jira defect reports covering 3 distinct defect scenarios (Calculation Overflow, Boundary Rejection, and Silent Validation Failure).

System Under Test Brief:

Target System: Retail Store Web Application. Defect 1: Coupon SAVE50 allows negative order total. Defect 2: Registration form rejects 18-year-old users. Defect 3: Phone number validation failure freezes form without error feedback.

Submission Instructions:

- Author at least 3 complete defect reports.
- Each defect report must contain all 9 required fields: Defect Title, Environment, Preconditions, Steps to Reproduce (numbered), Expected Result, Actual Result, Severity, Priority, and Evidence description.
- Demonstrate correct Severity vs Priority assignment for each defect.

Deterministic Evaluation Rubric:

- **[Weight 30%] At Least 3 Defect Reports [Requires at least 3]** Learner documents at least 3 distinct defect reports.
- **[Weight 40%] Complete Jira Standard Fields** Reports include title, environment, preconditions, steps, expected, actual, severity, priority, evidence.
- **[Weight 30%] Accurate Severity & Priority Classification** Demonstrates professional distinction between technical impact and business urgency.

5. Industry Interview Questions & Model Answers

Q1 [Difference-Based]: What is the difference between Retesting and Regression Testing?

Model Answer:

Retesting verifies that a specific defect has been fixed by executing the exact test cases that previously failed. Regression testing verifies that the new bug fix or code change did not break existing, unchanged features elsewhere in the application.

Avoid this mistake: Saying retesting and regression testing are the same thing.

Q2 [Scenario]: Can you describe a scenario where a defect has High Severity but Low Priority?

Model Answer:

A classic example is an application crash that occurs in an annual tax archiving module executed only on December 31st. Because it crashes the system, its Severity is High (Critical technical impact), but because the event is 8 months away and does not block daily users, its Priority is Low compared to urgent checkout issues.

Avoid this mistake: Assuming all high severity bugs must be fixed immediately within 1 hour.

Q3 [Workplace Behavioral]: What makes a bug report "good" from a developer perspective?

Model Answer:

A good bug report allows the developer to reproduce the issue without asking questions. It features a clear title ([Component] Summary under condition), exact environment details, numbered step-by-step reproduction instructions, expected vs actual behavior, and concrete evidence such as screenshots, console errors, or network payloads.

Avoid this mistake: Omitting reproduction steps and just submitting a screenshot.

6. Key Takeaways & Fast Revision Notes

Core Takeaways:

- Pass, Fail, Blocked, and Not-Run represent the four primary test execution outcomes.
- A defect moves through a structured life cycle from New to Closed.
- Severity measures technical impact; Priority measures business urgency.
- Always follow a bug fix with both Retesting (defect fix) and Regression Testing (side-effects).

Cheat Sheet / Revision Notes:

- Defect States: New -> Assigned -> Open -> Fixed -> Pending Retest -> Closed / Reopened.
- Severity Levels: Blocker (system down), Critical (core function broken/data loss), Major (feature impaired with workaround), Minor (cosmetic/typo).
- Smoke = broad & shallow (stability); Sanity = narrow & deep (post-fix verification).